Evaluating Symbolic Coordinate Discovery as a Robust Exploration Metric in Noisy Environments

CSI 5340 — Reinforcement Learning
University of Ottawa
Winter 2026

Edward He
University of Ottawa
ehe058@uottawa.ca

Haozheng Wang
University of Ottawa
hwang137@uottawa.ca

Sree Sahithya T.R.
University of Ottawa
stala099@uottawa.ca

Xujia Fan
University of Ottawa
xfan022@uottawa.ca

April 9, 2026

Abstract

In this project, we study how different exploration strategies affect reinforcement learning performance in Pokémon Red, a long-horizon and sparse-reward environment. We compare three approaches: a pixel-based KNN curiosity baseline (V1), a symbolic coordinate-based method (V2), and our extended model (V3), which adds memory, text decoding, and structured map reasoning.

We find that pixel-based exploration is highly unstable due to visual noise, while coordinate-based exploration provides a more reliable and efficient signal. Our V3 model further improves the agent's ability to explore by incorporating LSTM memory and semantic rewards. However, these improvements come with increased computational cost and introduce new training challenges.

Overall, our results show that exploration design and reward shaping have a stronger impact on performance than model complexity in this type of environment. https://github.com/EDHE08232001/PokemonRL.

Keywords: Reinforcement Learning, Proximal Policy Optimization, LSTM, Noisy TV Problem, Symbolic Exploration, Pokémon Red

Contents

1 Introduction

A central challenge in deep reinforcement learning (DRL) is effective exploration in environments with sparse extrinsic rewards and high-dimensional observation spaces. In the *Pokémon Red* domain, meaningful extrinsic rewards—event flags, gym badges, party level gains—fire at rare milestones separated by tens of thousands of optimal steps. Between these milestones, the agent receives zero gradient signal and idles indefinitely without an internal curiosity drive.

Intrinsic motivation mechanisms address this by providing a continuous learning signal that incentivizes visiting novel states. The state-of-the-art approach by Whidden [1] uses Pixel-KNN novelty: downsampled 36×40 RGB frames are stored in an HNSW index of up to 20,000 embeddings, and KNN distance serves as the exploration reward. While innovative, this approach suffers from the **"Noisy TV" problem** [1]: *Pokémon Red* is visually non-stationary, with grass animations, water ripples, and battle flashes creating endless "novel" frames without geographic movement. The agent stalls in visually dynamic areas, achieving zero progress while accumulating unbounded curiosity reward—analogous to staring at a television displaying random static.

Beyond the Noisy TV trap, pixel-based curiosity imposes massive computational overhead. Maintaining a KNN index of 20,000 frames $\times$ 4,320 floats per frame consumes significant CPU and RAM, limiting V1 to approximately 318 FPS with only 16 parallel environments. This makes rigorous multi-seed statistical evaluation practically infeasible on consumer hardware.

We address these limitations through a systematic three-paradigm evaluation:

(i) **V1 (Pixel-KNN Baseline):** Reproduces Whidden's approach to establish baseline behaviour and quantify the Noisy TV problem.

(ii) **V2 (Symbolic Coordinate Discovery):** Replaces pixel curiosity with deterministic `(X, Y, MapID)` coordinate counting via direct RAM reads, eliminating visual noise susceptibility.

(iii) **V3 (Semantic Reasoning):** Our original contribution—augments V2 with LSTM working memory, WRAM text decoding for narrative awareness, and topological graph navigation for structured map exploration.

Our primary research question is: *To what extent does a low-dimensional symbolic exploration reward (coordinate discovery) improve sample efficiency and reduce computational latency compared to high-dimensional pixel-based KNN curiosity under a matched compute budget?*

We evaluate along three axes: (1) sample efficiency measured by unique `(X, Y, MapID)` states visited per compute unit, (2) computational latency measured by per-step CPU/RAM overhead, and (3) matched compute budget using identical wall-clock time for fair cross-paradigm comparison.

2 Related Work

2.1 Reinforcement Learning in Game Environments

Training RL agents in complex game environments has been a productive testbed for algorithm development. Pokémon Red, a 1996 Game Boy RPG with combinatorial state spaces, sparse rewards, and long-horizon dependencies, presents unique challenges for exploration. Whidden [1] demonstrated the feasibility of training a Proximal Policy Optimization (PPO) agent to play Pokémon Red using pixel-based KNN curiosity, attracting significant community interest. Pleines et al. [2] extended this line of work with systematic evaluation of RL approaches in the Pokémon Red domain, providing additional baselines and analysis.

2.2 Proximal Policy Optimization

PPO [3] is an on-policy actor-critic algorithm that constrains policy updates via a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(r_t(\theta)\, \hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\, \hat{A}_t \Big) \right], \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and ϵ is the clipping parameter. PPO's simplicity and stability make it well-suited for complex game environments where training runs span millions of timesteps.

2.3 The Noisy TV Problem

The "Noisy TV" problem describes a failure mode of curiosity-driven exploration where an agent becomes trapped by a source of perpetual visual novelty [1]. In *Pokémon Red*, grass animations, water ripples, and battle flashes produce visually unique frames on every timestep. A pixel-based curiosity agent receives infinite intrinsic reward from standing in grass—every frame appears "novel" in pixel space—while making zero geographic progress. The name derives from the analogy of an agent staring at a television showing random noise: every frame is maximally novel, but no useful information is gained.

2.4 Intrinsic Motivation and Symbolic Representations

Intrinsic motivation provides a continuous learning signal between sparse extrinsic rewards. While most approaches operate in learned latent spaces (e.g., ICM, RND), our work investigates whether *symbolic* state representations—specifically, RAM-extracted game coordinates—can serve as a more robust and computationally efficient basis for curiosity. This low-dimensional, deterministic approach sidesteps the representational challenges that make pixel-based curiosity vulnerable to visual noise.

3 Methodology

3.1 Environment

All experiments use `RedGymEnv`, a custom OpenAI Gymnasium environment wrapping the PyBoy Game Boy emulator. The agent interacts with *Pokémon Red* at an action frequency of 24 (one action per 24 emulated frames). The game scope covers Pallet Town through Viridian Forest, encompassing approximately 15 accessible map regions with high-frequency area transitions.

3.1.1 Extrinsic Reward Signals

All three versions share a common set of extrinsic reward signals read from Game Boy RAM:

- **Event Flags:** Memory range `0xD747`–`0xD886` (∼1,720 flag bits). Each flipped bit indicates a story milestone, rewarded at +2 per event.
- **Gym Badges:** RAM address `0xD356`, a single byte with 8 bits (one per badge). Extremely rare.
- **Party Levels:** Sum of all party Pokémon levels, read from RAM.
- **Healing:** HP recovery at Pokémon Centers.
- **Death Penalty:** Negative reward on full party blackout.

3.2 V1: Pixel-KNN Baseline

V1 reproduces Whidden's pixel-based curiosity approach [1].

Observation Space. Downsampled 36×40 RGB frames yield 4,320 floats per observation, fed into a `CnnPolicy` (Nature CNN: 3 convolutional layers followed by a linear head).

Intrinsic Reward. An HNSW (Hierarchical Navigable Small Worlds) index [1] stores up to 20,000 frame embeddings. KNN distance to the current frame serves as the exploration reward—unbounded and noisy.

Reward Formula.

$$R_{\text{V1}} = r_s \times \big(1 \cdot e + 1 \cdot l + 1 \cdot h + 1 \cdot o + (-0.1) \cdot d + 5 \cdot b + \text{KNN}_{\text{explore}}\big), \tag{2}$$

where $r_s = 4$ (reward scale), e = event flags, l = levels, h = healing, o = opponent level, d = death, b = badges. All extrinsic multipliers are flat ($1\times$) except badges ($5\times$) and death (-0.1). The KNN exploration term is raw distance-based with no cap.

Training Configuration. `CnnPolicy`, 16 parallel environments, `n_epochs`= 3, `batch_size`= 128, $\gamma = 0.998$, `reward_scale`= 4, `explore_weight`= 3.

3.3 V2: Symbolic Coordinate Discovery

The goal of V2 is to replace noisy pixel-based exploration with a stable and deterministic coordinate-based approach. Instead of relying on visual similarity, the agent directly tracks its position in the game world using RAM values. V2 replaces pixel-based curiosity with symbolic coordinate counting.

Coordinate Counting. The player's position is read directly from RAM addresses `0xD361` (Y), `0xD362` (X), and `0xD35E` (Map ID). Each unique `(X, Y, MapID)` tuple is stored in a hash map. The intrinsic reward scales linearly with the number of unique coordinates discovered.

Observation Space. A multi-input architecture (`MultiInputPolicy`) processes:

- `screen`: $72 \times 80 \times 1$ grayscale Game Boy screen
- `hp_fraction`: scalar, current HP / max HP for the lead Pokémon
- `level`: 8 Fourier features computed as $\sin(\text{level} \times 2^i)$ for $i \in [0, 8)$
- `badges`: 8-dimensional binary vector (one bit per gym badge)

Fourier Encoding. A raw scalar level value (e.g., 15) provides poor gradient signal to a neural network. Fourier encoding projects it into an 8-dimensional space:

$$\text{fourier}(v) = [\sin(v \cdot 2^0),\ \sin(v \cdot 2^1),\ \ldots,\ \sin(v \cdot 2^7)], \tag{3}$$

where low frequencies $(2^0, 2^1)$ capture coarse magnitude and high frequencies $(2^6, 2^7)$ capture fine-grained differences. This is analogous to positional encoding in Transformers.

Stuck Penalty. When the agent visits the same `(X, Y, MapID)` tile for 600 or more consecutive timesteps, a penalty of -0.05 is applied. At `action_freq`$= 24$, this corresponds to $600 \times 24 = 14{,}400$ Game Boy frames ($\approx$4 minutes of real game time), distinguishing genuine stuck behaviour from intentional stationary actions such as battling or healing.

Reward Formula.

$$R_{\text{V2}} = r_s \times \big(4 \cdot e + 10 \cdot h + 10 \cdot b + e_w \times |\text{coords}| \times 0.1 - 0.05 \cdot \text{stuck}\big), \tag{4}$$

where $r_s = 0.5$, $e_w = 0.25$. Note that level and opponent-level terms are absent in V2.

Training Configuration. `MultiInputPolicy`, 64 parallel environments, `n_epochs`$= 1$, `batch_size`$= 512$, $\gamma = 0.997$, `ent_coef`$= 0.01$.

3.4 V3: Semantic Reasoning (Our Contribution)

V3 extends V2 by introducing memory, semantic understanding, and structured navigation. The main idea is to move beyond simple spatial exploration and allow the agent to make more informed decisions over time. V3 extends V2 with three new capabilities: LSTM working memory, WRAM text decoding, and topological graph navigation.

3.4.1 LSTM Working Memory

V3 uses `MultiInputLstmPolicy` with 128 hidden units and 1 layer. The persistent LSTM hidden state replaces frame stacking (`frame_stacks`= 1), enabling multi-step sequential reasoning across timesteps. At each step, the LSTM receives the concatenated feature vectors from all input streams and updates its internal representation of recent history.

3.4.2 Semantic Text Decoding

V3 hooks into WRAM address `0xCF4B`, a 20-byte active text buffer, and decodes it using the original Gen 1 character map:

- Uppercase A–Z: bytes `0x80`–`0x99`
- Lowercase a–z: bytes `0x9A`–`0xB3`
- Digits 0–9: bytes `0xF6`–`0xFF`
- Terminator: `0x50`

Each decoded string is hashed via MD5 to produce a fixed 8-byte observation vector. A one-time reward of +0.5 is granted for each novel dialogue string discovered. Collisions are astronomically unlikely for the ∼200 unique dialogue strings in early-game *Pokémon Red*.

3.4.3 Topological Graph Navigation

V3 constructs a `networkx.DiGraph` at runtime:

- **Nodes:** Every unique `(X, Y, MapID)` coordinate becomes a node.
- **Movement edges:** Adjacent tile transitions create directed edges.
- **Warp edges:** When `map_id` changes between consecutive steps (doors, ladders, cave entrances), a warp edge is logged.
- **Distance metric:** Shortest path from spawn to the furthest discovered node, cached in a dictionary for $O(1)$ per-step lookups.
- **New map bonus:** +2.0 reward for entering any previously unseen map region—the highest single intrinsic signal in V3.

3.4.4 V3 Observation Space

V3 inherits V2's observation space and adds:

- `map_crop`: local map region around the player
- `text_hash`: 8 bytes from MD5 hash of decoded WRAM text buffer
- `graph_distance`: scalar, shortest topological distance to furthest node (cached $O(1)$)

All inputs pass through feature extractors, are concatenated, and fed into the LSTM (128 hidden, 1 layer), which outputs both a policy head and a value head.

3.4.5 V3 Stuck Penalty

V3 uses a stronger stuck penalty of −0.5 (10× that of V2) under the same 600-consecutive-visit condition. Critically, the penalty is **suppressed when `text_hash` is nonzero** (i.e., NPC dialogue is active), preventing the agent from being penalized for standing still while reading dialogue—which is intentional and rewarded behaviour.

3.4.6 V3 Reward Formula

$$\begin{aligned} R_{\text{V3}} = r_s \times \big(&4e + f(l) + 10h + 25b + 0.2o + e_w \cdot |\text{coords}| \cdot 0.1 \\ &+ \text{stuck} + 2.0 \cdot \text{map} + \text{sem} + 3.0 \cdot \text{party} + 5.0 \cdot \text{oak} \\ &+ 0.5 \cdot (\text{battle\_entry} + \text{enemy\_faint} + \text{trainer\_win})\big), \end{aligned} \tag{5}$$

where $r_s = 0.5$, $e_w = 1.0$ (post-fix). Level scaling applies a diminishing function: $f(l) = l$ if $l \leq 22$; $f(l) = 22 + (l - 22)/4$ otherwise. Semantic rewards include +0.5 per new dialogue and +2.0 per new map. Party growth uses diminishing returns: $+2.5, +2.0, +1.5, +1.0, +0.5, +0.5$ for each additional Pokémon. Oak's parcel quest milestones earn +5.0 each.
This indicates that adding memory and semantic signals improves the agent's capability, but also increases training complexity and reduces throughput.

3.5 Training Configuration Comparison

Table 1 summarizes the complete training configuration across all three paradigms.

Table 1: Training configurations across V1, V2, and V3 (post-fix values for V3).

Parameter	**V1**	**V2**	**V3 (post-fix)**
Policy	CnnPolicy	MultiInputPolicy	MultiInputLstmPolicy
LSTM size	—	—	128 hidden, 1 layer
Num envs	16	64	64
`n_epochs`	3	1	1
`batch_size`	128	512	512
γ	0.998	0.997	0.997
`ent_coef`	0.0 (default)	0.01	0.02
`vf_coef`	0.5 (default)	0.5 (default)	0.75
`clip_range`	0.2 (default)	0.2 (default)	0.15
`reward_scale`	4	0.5	0.5
`explore_weight`	3	0.25	1.0
`action_freq`	24	24	24
`frame_stacks`	default	default	1

3.6 Compute Infrastructure

All training runs were executed on the Morningstar HPC cluster at the University of Ottawa via SLURM job scheduling. V1 was trained locally due to its lower parallelism requirements. V2 and V3 runs utilized cluster GPU nodes with 64 parallel environments managed by `SubprocVecEnv`.

4 Experimental Results

4.1 V1 Baseline Results

The V1 baseline confirms the Noisy TV problem empirically. Training reward exhibits extreme instability, swinging from $20.39 \rightarrow 58.70 \rightarrow 72.49 \rightarrow 23.49$ across checkpoints. The peak reward

of 72.49 is illusory: exploration reward spikes to 19.87, driven by visual novelty from grass and water frames rather than geographic progress. When visual novelty saturates, both exploration and total reward collapse. Key observations:

- **Training instability:** Reward swings of $> 3\times$ between checkpoints.
- **Explore collapse:** Exploration reward oscillates erratically ($4.67 \rightarrow 19.87 \rightarrow 5.87$).
- **Throughput:** $\sim$318 FPS, limited by KNN index CPU cost with only 16 environments.
- **Reproducibility:** High variance across seeds makes statistical evaluation impractical.

4.2 V2 Results

V2 eliminates the Noisy TV problem entirely through symbolic coordinate counting. Key metrics:

- **Throughput:** 1,040–1,113 FPS (3.3$\times$ faster than V1), enabling 64 parallel environments.
- **Approx KL:** 0.005–0.007 (tight policy updates).
- **Clip fraction:** 0.04–0.08 (well within trust region).
- **Explained variance:** Converges from -0.12 to $\sim$0.9 (the value network accurately predicts returns).
- **Value loss:** Converges to near zero.

The deterministic, noise-immune nature of coordinate counting produces stable training with low inter-seed variance. The $\sim 10\times$ lower memory footprint compared to V1's KNN index enables significantly more parallel environments.

4.3 V3 Results: 10.5M Timesteps

At 10.5M timesteps, V3 demonstrates strong convergence across all key metrics:

- **Value loss:** Drops from 0.033 to $\sim$0.002 within 1.5M steps and remains near-zero for the remaining 9M+ steps.
- **Explained variance:** Peaks at 0.76, indicating the critic predicts returns well.
- **Approx KL:** $\sim$0.013, slightly higher than V2 due to the richer observation space.
- **Throughput:** Stable at $\sim$264 FPS ($\sim 4\times$ slower than V2 due to LSTM recurrence).

The $4\times$ throughput reduction relative to V2 is a deliberate trade-off: every forward pass must carry and update LSTM hidden and cell state tensors of shape $(1, 128)$. At 264 FPS $\times$ 64 environments, the system still processes $\sim$16,896 frames per second in aggregate.

Exploration Achievements. The best V3 run reached Viridian Forest, discovering:

- 15 maps (Pallet Town, Route 1, Viridian City, Route 2, Viridian Forest, plus interiors)
- 1,772 unique `(X, Y, MapID)` tiles
- 19 unique NPC dialogues decoded and rewarded
- 1,772 topological graph nodes

Averaged across all 64 parallel environments: 9.4 mean maps and 1,171 mean tiles.

4.4 V2 vs. V3: Comparative Analysis

Table 2 presents the head-to-head comparison.

Table 2: Comparative analysis of V2 (Symbolic) and V3 (Semantic Reasoning).

Metric	V2 (Symbolic)	V3 (Our Work)	Winner
Training FPS	1,040–1,113	256–270 (∼264 avg)	V2
Value Loss	→ near zero	→ 0.002 (1.5M steps)	Tie
Explained Variance	−0.12 → ∼0.9	0.09 → 0.76 peak	Tie
Curiosity Scope	Spatial only	Spatial + Semantic + Map	V3
Memory Architecture	Frame stacking	LSTM hidden state (128h)	V3
KL Divergence	0.005–0.007	0.011–0.017	V2
Novel Rewards	Coord count only	+Dialogue +Map discovery	V3

V3 trades throughput for richer cognitive exploration—a deliberate research trade-off targeting semantic understanding. V2 maintains tighter KL divergence because its simpler observation space yields a more constrained optimization landscape.

5 Extended Training and Regression Analysis

5.1 V3 Extended Training: 84M Steps

To evaluate V3's long-horizon behaviour, we extended training to 84M timesteps across three SLURM jobs on the Morningstar HPC cluster:

1. **Job 4867** (31.5M → 41.9M): Initial training phase with gradual improvement.
2. **Job 4870** (42M → 62M): Peak performance—average reward reached 23.55 with the strongest exploration behaviour.
3. **Job 4873** (63M → 83.9M): Critical regression—reward crashed from 23.55 to 2.63 at checkpoint resume, eventually recovering to 19.36. An 18% regression versus peak.

The most striking finding: **zero gym badges across all 84M steps**. Despite discovering over 1,600 unique tiles, the agent never defeated a gym leader. Badge reward was dwarfed by exploration reward, which constituted approximately 40% of total reward. The agent was rationally optimizing what the reward function incentivized—exploration was more profitable than battling.

5.2 Diagnosing the Regression

Post-mortem analysis of TensorBoard diagnostics identified four root causes:

Root Cause 1: Value Network Collapse. Explained variance degraded from 0.39 to 0.16. The primary culprit was `n_epochs`= 3: in standard (non-recurrent) PPO, reusing a rollout for multiple gradient epochs is safe because observations are static tensors. However, with an LSTM, hidden states computed in epoch 1 become *stale* by epoch 2 because the policy weights have already been updated. The LSTM processes states generated by an older version of itself, leading to inconsistent value estimates and destabilizing updates. **This is the most critical finding of our regression analysis.**

Root Cause 2: Entropy Collapse. Entropy degraded from -1.94 to -1.62. The policy lost exploratory diversity, converging on a narrow strategy. Lower entropy means fewer distinct actions are tried, and the agent never discovers gym-battle behaviour.

Root Cause 3: Reward Imbalance. At 1,600 tiles $\times$ 0.1 reward per tile = 160 accumulated exploration reward versus one badge at $10 \times 0.5 = 5$ reward, exploration was approximately 32$\times$ more rewarding than badge pursuit. Additionally, the opponent-level reward (`op_lvl`) had been accidentally dropped from V3's reward function, removing a key incentive for battle engagement.

Root Cause 4: KL/Clip Instability. Clip fraction doubled from 0.06 to 0.11, indicating the policy was overshooting its trust region on approximately 11% of update steps.

Key insight: The agent was not failing—it was optimizing exactly what we told it to optimize. The regression was a consequence of reward design and hyperparameter interaction, not algorithmic insufficiency.

5.3 Hyperparameter Fixes

Based on the regression diagnosis, we applied six targeted fixes, summarized in Table 3.

Table 3: Hyperparameter fixes applied to V3 (committed to GitHub).

Parameter	Before	After	Rationale
`n_epochs`	3	1	Fix LSTM stale hidden states (most critical)
`ent_coef`	0.01	0.02	Prevent entropy collapse
`vf_coef`	0.5	0.75	Strengthen value network (compensate fewer gradient steps)
`clip_range`	0.2	0.15	Tighter trust region
γ	0.995	0.997	Longer reward horizon ($\frac{1}{1-\gamma}$: 200 $\rightarrow$ $\sim$333 steps)
`badge_mult`	10	25	Prioritize badge pursuit ($25 \times 0.5 = 12.5$ vs. $10 \times 0.5 = 5.0$)
`op_lvl` reward	Off	On	Incentivize battle training (+0.2$\times$ max opponent level)

Rationale for `n_epochs` 3 $\rightarrow$ 1. This is the single most impactful fix. With recurrent policies, multi-epoch rollout reuse corrupts LSTM hidden states because the policy weights update between epochs while the stored hidden states remain frozen from the original forward pass. Setting `n_epochs`= 1 ensures each trajectory is processed exactly once with fresh context. The trade-off—fewer gradient steps per rollout—is compensated by increasing `vf_coef`.

Rationale for γ 0.995 $\rightarrow$ 0.997. The discount factor determines the effective planning horizon: $\frac{1}{1-\gamma}$. At $\gamma = 0.995$, the horizon is 200 steps; at $\gamma = 0.997$, it extends to $\sim$333 steps. Since badge rewards are temporally distant (requiring thousands of optimal steps), a longer horizon helps these rewards propagate backward through the value function.

Rationale for `clip_range` $0.2 \rightarrow 0.15$. PPO clips the probability ratio $r_t(\theta)$ to $[1-\epsilon, 1+\epsilon]$. Reducing ϵ from 0.2 to 0.15 makes updates more conservative, directly addressing the clip fraction overshoot observed in the regression ($0.06 \rightarrow 0.11$).

5.4 Post-Fix Extended Training: 157M Steps

Following the hyperparameter fixes, training continued from 125.8M to 157.3M timesteps across two SLURM jobs:

Job 4894 (125.8M → 146.8M). Training stabilized with the corrected configuration. Mean reward settled at ∼19.4, with exploration dominating the reward composition. Zero badges were obtained. The agent exhibited stable but plateau-like behaviour.

Job 4901 (146.8M → 157.3M). With `clip_range`= 0.15 and `explore_weight`= 1.0:

- **Explained variance:** ∼0.72, confirming the value network fix was effective.
- **Value loss:** ∼0.002, near-optimal.
- **Badges:** Still zero across all 157M+ steps.
- **Behaviour:** Stable training dynamics with no regression, but the agent remained stuck in a local optimum—exploration reward continued to dominate over battle engagement.

The post-fix results confirm that the hyperparameter corrections successfully eliminated the regression and stabilized training. However, the zero-badge outcome across 157M+ steps indicates that the fundamental reward imbalance—intrinsic exploration reward dominating extrinsic badge reward—requires further architectural or curriculum-based interventions beyond hyperparameter tuning.

6 Discussion

6.1 Symbolic Exploration Eliminates the Noisy TV Problem

Our results provide strong empirical evidence that symbolic coordinate discovery is a robust alternative to pixel-based curiosity in visually non-stationary environments. V2's deterministic `(X, Y, MapID)` counting is completely immune to visual entropy—grass animations, water ripples, and battle flashes cannot generate spurious novelty signals. The 3.3× throughput improvement (1,040–1,113 FPS vs. 318 FPS) enables 10-seed reproducible benchmarks on consumer hardware, a capability infeasible with V1's pixel-KNN approach.

6.2 LSTM Recurrence Requires Single-Epoch PPO

The most impactful finding from our regression analysis is that **multi-epoch PPO rollout reuse is incompatible with recurrent policies**. In standard PPO, reusing rollout data for multiple gradient epochs improves sample efficiency. However, with an LSTM, hidden states computed during the initial forward pass become stale after the first gradient update. Subsequent epochs process these frozen hidden states with updated weights, creating a distributional mismatch that destabilizes the value network and degrades policy quality. This finding has broad implications for any practitioner using recurrent architectures with PPO.

6.3 Reward Design as the Binding Constraint

Despite 157M+ timesteps of training with corrected hyperparameters, the V3 agent obtained zero gym badges. The agent was not failing—it was optimally exploiting the reward function as designed. Exploration reward accumulated at $\sim$0.1 per new tile across $>$ 1,600 tiles, vastly outweighing the one-time badge reward of $25 \times 0.5 = 12.5$. This highlights a fundamental challenge in intrinsic motivation: when exploration is sufficiently rewarding, agents have no incentive to pursue sparse extrinsic objectives.

6.4 Throughput vs. Cognitive Depth Trade-Off

V3's $\sim$4$\times$ throughput reduction relative to V2 (264 vs. 1,040+ FPS) is an inherent cost of LSTM recurrence, as each forward pass must carry and update hidden and cell state tensors. For domains where sequential reasoning and narrative awareness are critical, this trade-off may be justified. For pure spatial exploration tasks, V2's symbolic coordinate counting offers a superior efficiency-to-performance ratio.

From a practical perspective, our experiments show that exploration strategy and reward design are the main factors that determine performance in long-horizon environments. Increasing model complexity alone does not guarantee better results.

7 Conclusion and Future Work

7.1 Core Contributions

1. **Symbolic coordinate reward eliminates the Noisy TV problem.** Deterministic `(X, Y, MapID)` counting is noise-immune and computationally lightweight.
2. **Low-latency RAM hooking enables reproducible benchmarks.** The 3.3$\times$ throughput improvement makes 10-seed statistical evaluation feasible on consumer hardware.
3. **Stuck penalty forces loop-breaking.** The -0.05 (V2) / -0.5 (V3) penalty with dialogue suppression mathematically prevents local minima traps.
4. **V3 LSTM + topological graph enables narrative-aware exploration.** Our original contribution combines working memory, text decoding, and map topology for cognitive depth.
5. **LSTM-specific PPO failure mode identified.** Multi-epoch rollout reuse corrupts recurrent hidden states—a practical guideline for the community.
6. **Map discovery heatmaps provide interpretable visualization.** Symbolic coordinates enable trajectory analysis across all three paradigms.

   We also observe that using multiple PPO epochs with recurrent policies can lead to unstable training due to stale hidden states, suggesting that single-epoch updates are more suitable for LSTM-based models.

7.2 Future Directions

1. **Scale V3 to full Kanto** with rigorous multi-seed evaluation across the complete game.

2. **Curriculum learning** for staged map unlocking and guided exploration, potentially bridging the gap between intrinsic exploration and extrinsic badge pursuit.
3. **Hybrid symbolic + latent embeddings** to combine the noise-immunity of symbolic coordinates with the representational richness of learned features.
4. **Reward shaping for badge pursuit,** including distance-based badge proximity rewards or hierarchical reward structures that explicitly prioritize battle engagement.
5. **Cross-domain generalization** to other games (Zelda, Minecraft) and robotics domains where structured symbolic exploration may outperform pixel-based curiosity.

References

[1] P. Whidden. Training AI to Play Pokemon with Reinforcement Learning. YouTube, 2023. https://youtu.be/DcYLT37ImBY.

[2] M. Pleines, J. Palomaki, F. Coste, and S. Risi. Pokemon Red via Reinforcement Learning. *arXiv preprint arXiv:2502.19920v2*, 2025. https://arxiv.org/abs/2502.19920.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017. https://arxiv.org/abs/1707.06347.

[4] Group 3. PokemonRL: Evaluating Symbolic Coordinate Discovery as a Robust Exploration Metric. GitHub Repository, University of Ottawa, 2026. https://github.com/EDHE08232001/PokemonRL.